

Parameterized design using VHDL

Lecture 8

10100101010011110100001001011101001011010101011101000041000101010100
004100001010010100100101000010110100101014000011110100101010011101000010010111010010
110101010101110100004100001010010100101000010110100101014000011110100101

Electronic and Communication Engineering
Department
ECE 307 Introductory to Digital Systems Lab
Dr. Ammar Jallawi Mahmood
Ammar.karkar@uokufa.edu.iq

Parameterizable VHDL

- Good (and useful) design practice for VHDL implies the use of *parameterization*
- Parameterization in VHDL implies that values such as array sizes, data widths, etc. should never be explicitly defined but should rather be set using a particular kind of constant called a **generic**

```
generic (par_name : par_type := par_value);
```

- Parameterized VHDL code should *synthesize correctly for any value of the parameters defined in generics*
- Generics are a kind of constant, but they cannot be declared in an architecture. Instead, they are *part of the entity declaration*:

```
entity my_entity is  
  generic (n : integer := 7);  
  port (...);  
end my_entity;
```

**Default value
(always required)**



Parameterizable VHDL

- Within the entity, however, generics *behave as constants*. They are very often used to define vector sizes (including port sizes). Example:

```
entity my_entity is
  generic (v1size : natural := 4,
           v2size : natural := 8);
  port (clk, rst, en : in std_logic;
        data_out : out signed (v2size-1 downto 0));
end my_entity;
```

Default value
(always required)



```
architecture arch of my_entity is
  signal v1: unsigned (v1size-1 downto 0);
  signal v2: unsigned (v2size-1 downto 0);
  [...]
```

- Note that generics are used for synthesis, but *are not themselves synthesized*! They are most often type **integer** or **natural**.

Parameterizable VHDL

- Note that *the initial values assigned to generics are default values that should always be overwritten* when the component is instantiated in another entity, using the **generic map** statement:

```
label: comp_name  
  generic map ([generics])  
  port map ([ports]);
```

- For example, if the component above were to be instantiated as:

```
ent_1: entity work.shreg  
  generic map (data_size => 5)  
  port map (...);
```

**ALWAYS USE
GENERIC MAP ON
PARAMETERIZABLE
COMPONENTS,
EVEN IF THE
DEFAULT IS OK!**

Then a 5-bit shift register would be synthesized, irrespective of the default generic value in the **shreg** entity. (Type and size must be coherent, though!).

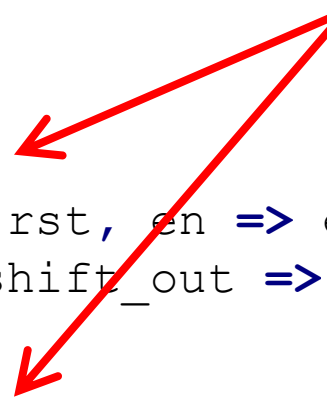
Generic components

- For example, suppose that we want to design a circuit that instantiates *two shift registers of different sizes*:

```
entity my_circuit is
    port (clk, rst, en : in STD_LOGIC;
          inp1, inp2: in STD_LOGIC;
          outp1, outp2: out STD_LOGIC);
end my_circuit;
-----
architecture circuit of my_circuit is

begin
r1: entity work.shreg
    generic map(data_size => 8 )
    port map(clk => clk, rst => rst, en => en,
             shift_in => inp1, shift_out => outp1);
r2: entity work.shreg
    generic map(data_size => 4 )
    port map(clk => clk, rst => rst, en => en,
             shift_in => inp2, shift_out => outp2);
end circuit;
```

These override the default size (32) and generate one 8-bit and one 4-bit shift registers



Generic components

- This works for *component instantiation* as well, but you should take care:

```
entity my_circuit is
    port (clk, rst, en : in STD_LOGIC;
          inp1, inp2: in STD_LOGIC;
          outp1, outp2: out STD_LOGIC);
end my_circuit;
-----
architecture circuit of my_circuit is
    component shreg is
        generic (data_size : integer);
        port (clk, shift_in, rst, en : in STD_LOGIC;
              shift_out : out STD_LOGIC);
    end shreg;
begin
    r1: shreg generic map(data_size => 8 )
        port map(clk => clk, rst => rst, en => en,
                  shift_in => inp1, shift_out => outp1);
    r2: shreg generic map(data_size => 4 )
        port map(clk => clk, rst => rst, en => en,
                  shift_in => inp2, shift_out => outp2);
end circuit;
```

The presence and type of generics must be included in the declaration

The values of the generics are set at instantiation (same as before)

Generic components

- While it can be, the "override" value should NOT be defined in the declaration (but always in the instantiation)

```
entity my_circuit is
    port (clk, rst, en : in STD_LOGIC;
          inp1, inp2: in STD_LOGIC;
          outp1, outp2: out STD_LOGIC);
end my_circuit;
-----
architecture circuit of my_circuit is
    component shreg is
        generic (data_size : integer := 8);
        port (clk, shift_in, rst, en : in STD_LOGIC;
              shift_out : out STD_LOGIC);
    end shreg;
begin
    r1: shreg
port map (clk => clk, rst => rst, en => en,
          shift_in => inp1, shift_out => outp1);
    r2: shreg
port map (clk => clk, rst => rst, en => en,
          shift_in => inp2, shift_out => outp2);
end circuit;
```

BAD

Overwrite value
should not be
in declaration

Generic components

- Generics are often *passed down in the component hierarchy*:

```
entity my_circuit is
    generic (r1_size : integer := 8;
             r2_size : integer := 4);
    port (clk, rst, en : in STD_LOGIC;
          inp1, inp2: in STD_LOGIC;
          outp1, outp2: out STD_LOGIC);
end my_circuit;
```

Top level
entities often
have several
generics

```
-----
architecture circuit of my_circuit is
```

```
begin
```

```
r1: entity work.shreg
    generic map(data_size => r1_size)
    port map(clk => clk, rst => rst, en => en,
             shift_in => inp1, shift_out => outp1);
```

```
r2: entity work.shreg
    generic map(data_size => r2_size)
    port map(clk => clk, rst => rst, en => en,
             shift_in => inp2, shift_out => outp2);
```

```
end circuit;
```


Generics and testbenches

- While of course parameterizable entities should always be verified using testbenches, it is very complex (and somewhat illogical) to design testbenches that are themselves parameterizable
- Testbenches should be applied to **one specific set of generic values for the UUT** (actually, to properly verify parameterization, more than one set should be used, with **a separate testbench for each set of generic values**)
- The values of the generics for the UUT should be defined as **constants** in the testbench (and not as generics)

Generics as compiler instructions

- *Generics are constants – usually integers (or even more often naturals)*
- *In that context, generics are a kind of **compiler instruction**, and are therefore **not synthesized**.*
- *As a consequence, they can be manipulated using **non-synthesizable functions**:*

```
-- LOG BASE 2 FUNCTION
-- returns the ceiling of log base 2 of a
-- (non-zero) integer
-- (1->0; 2->1; 3->2; 4->2; 5->3 ...)
```

```
function log2 ( x : natural ) return natural;
```

```
-- SIZE FUNCTION
-- returns the size of a vector that can
-- encode a (non-zero) integer
-- (1->1; 2->2; 3->2; 4->3; 5->3 ...)
```

```
function size ( x : natural ) return natural;
```

Example: a parameterizable counter

```
use work.DigEng.all;
```

Package containing the log2 and size functions

```
-- counter to LIMIT (0 to LIMIT-1) with synchronous reset and enable
```

```
entity Param_Counter is
```

```
generic (LIMIT : NATURAL := 17);
```

Counter limit defined by a generic (natural)

```
port ( clk, rst, en : in STD_LOGIC;
```

```
      count_out : out UNSIGNED (log2(LIMIT)-1 downto 0));
```

```
end Param_Counter;
```

```
architecture Behavioral of Param_Counter is
```

```
    signal count_int : UNSIGNED (log2(LIMIT)- 1 downto 0);
```

```
begin
```

```
counter: process (clk)
```

```
begin
```

```
    if rising_edge(CLK) then
```

```
        if (rst = '1') then
```

```
            count_int <= (others => '0');
```

```
        elsif (en = '1') then
```

```
            if (count_int = LIMIT-1) then
```

```
                count_int <= (others => '0');
```

```
            else
```

```
                count_int <= count_int + 1;
```

```
            end if;
```

```
        end if;
```

```
    end if;
```

```
end process counter;
```

```
count_out <= count_int;
```

```
end Behavioral;
```

What is the size of these vectors?

Counter resets when limit is reached

The log2 function is NOT synthesizable – it is used to compute a constant value for synthesis

Generics in synthesizable code

- Generics are constants – usually integers (or even more often naturals)
- For the most part, generics are used to determine **vector (bus) sizes** or to control **for-generate loops**
- In some rare cases, however, generics need to be used **within synthesizable code**.
- If that is the case, two scenarios are common:
 1. They are used in **tests and comparisons**, in which case using signed/unsigned signals is sufficient

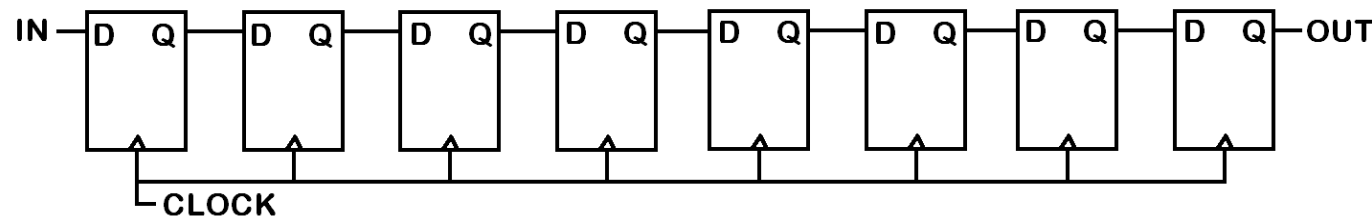
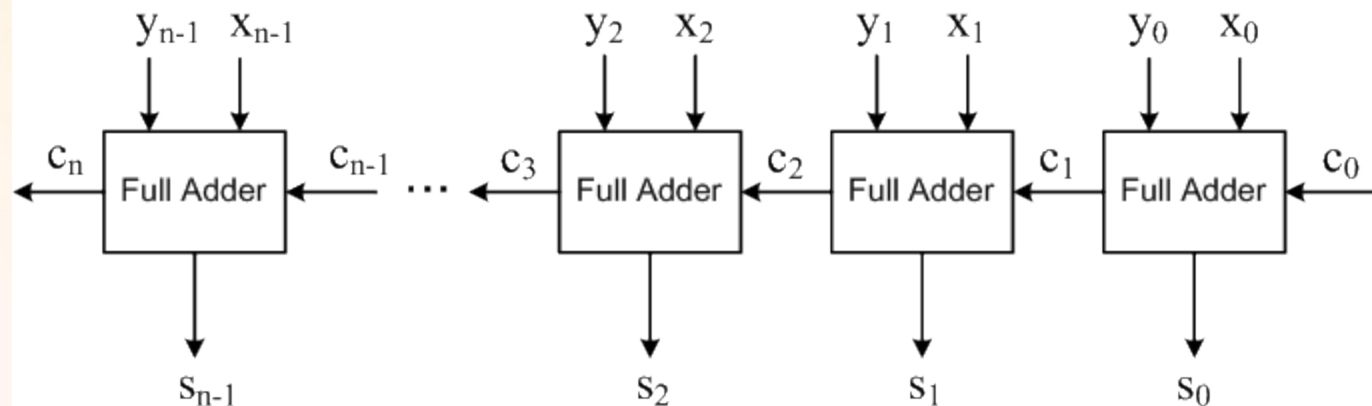
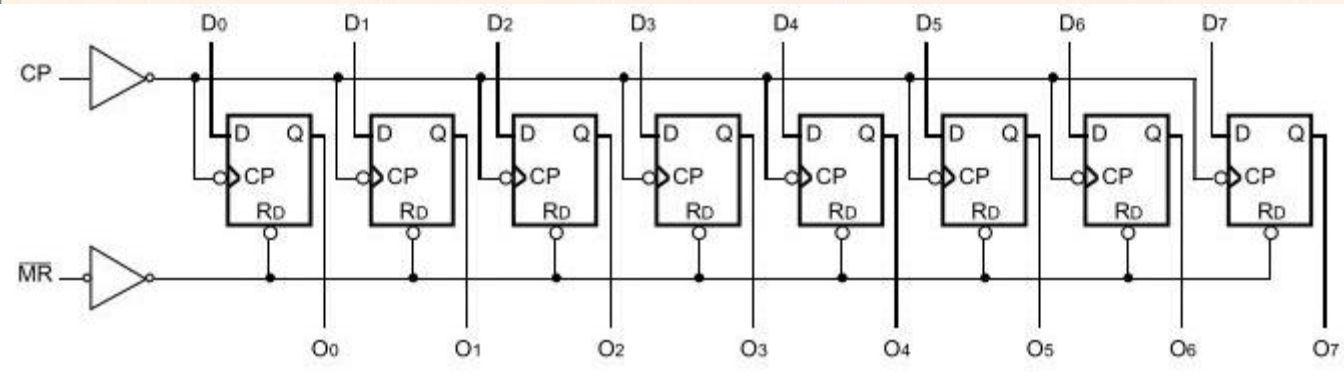
```
generic (LIMIT : NATURAL := 17);  
[...]  
    if (count_int = LIMIT-1) then
```

2. They are used in **assignments**, in which case, to avoid "nasty" (hard to detect) errors, it is good coding practice to **convert them into signals**

```
generic (OFFSET : NATURAL := 255);  
[...]  
offset_value <= base_value + to_unsigned(OFFSET, size(OFFSET));
```

Iterative logic structures

- In digital logic there are many repetitive or **iterative** structures*



Iterative logic structures

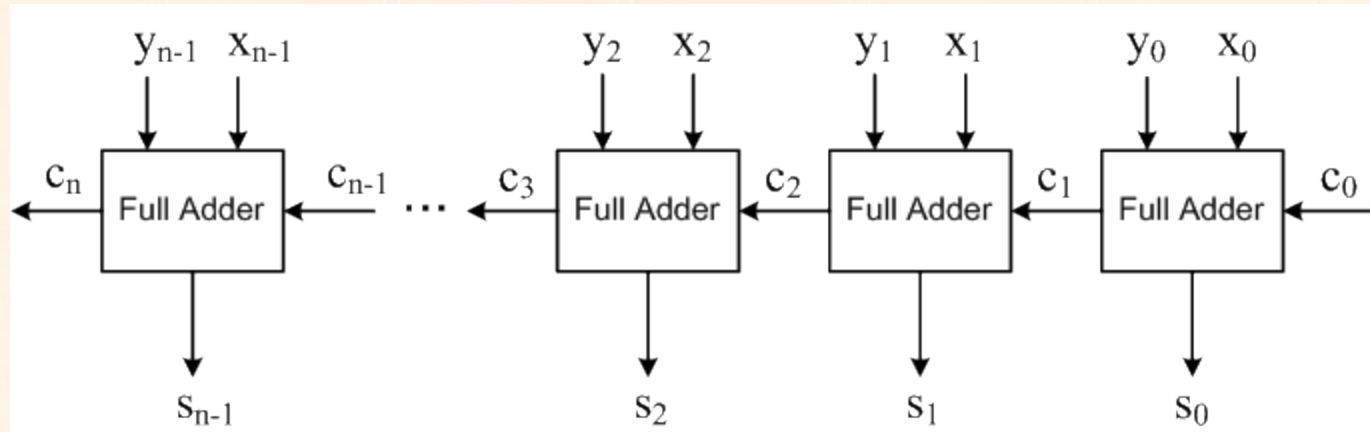
- In some (many?) cases, this can be handled implicitly using vectors instead of single bits*

```
entity REG_8BIT is
  port (CLK, RST, WEN : in STD_LOGIC;
        Data: in STD_LOGIC_VECTOR(7 downto 0);
        Q : out STD_LOGIC_VECTOR(7 downto 0));
end REG_8BIT;
```

```
architecture arch of REG_8BIT is
begin
  REG8: process (CLK)
  begin
    if (rising_edge(CLK)) then
      if (RST = '1') then
        Q <= (others => '0');
      elsif (WEN = '1') then
        Q <= Data;
      end if;
    end if;
  end process REG8;
end arch;
```


Iterative logic structures

- In other cases, particularly when components are involved, this is not directly possible*



- For such cases, VHDL supports (somewhat) the use of **loops***
- However, a distinction must be made:*
 - General-purpose loops** are (mostly) used in non-synthesizable code (mainly testbenches)*
 - The **for-generate loop** is specifically designed for synthesis*

Loops

- *General-purpose loops* are sequential statements (i.e., they can only be used inside processes) that.
- There are three kinds of loop statement in VHDL:
 - `while` [condition] `loop`
 - `for` [range] `loop`
 - `loop` `-- infinite loop`
- `For` loops (and in some rare cases `while` loops as well), can be synthesizable, but only if some *very strict rules* are followed
- However, loops can be extremely useful in *testbenches*

For Loops

- *Syntax:*

```
[opt_label :] for parameter in range loop  
    [sequential statements]  
    [next [opt_label] [when condition];  
    [exit [opt_label] [when condition];  
end loop[ opt_label];
```

- The **for** loop defines a *loop parameter* which takes on the type of the range specified. For example, the range “0 to 3” implies an integer: it does not need to be declared.

```
process (A)  
begin  
    Z <= "0000";  
    for i in 0 to 3 loop  
        if (A = i) then Z(i) <= '1';  
        end if;  
    end loop;  
end process;
```

- The range may be any discrete range, including an enumerated type, but the *limits must be static*

While and Infinite Loops

- *Syntax:*

```
opt_label: while [cond] loop  
    [sequential statements]  
end loop;
```

```
opt_label: loop  
    [sequential statements]  
end loop;
```

- *Almost invariably used only in testbenches (see infinite clock loop for example) and not for synthesis*
- *The **while** loop repeats a sequence of statements if a condition is true. The condition is tested before the iterations.*

```
process (A)  
    variable I :  
        integer range (0 to 4) := 0;  
begin  
    Z <= "0000";  
    while (I <= 3) loop  
        if (A = I) then Z(I) <= '1';  
        end if;  
        I := I + 1;  
    end loop;  
end process;
```

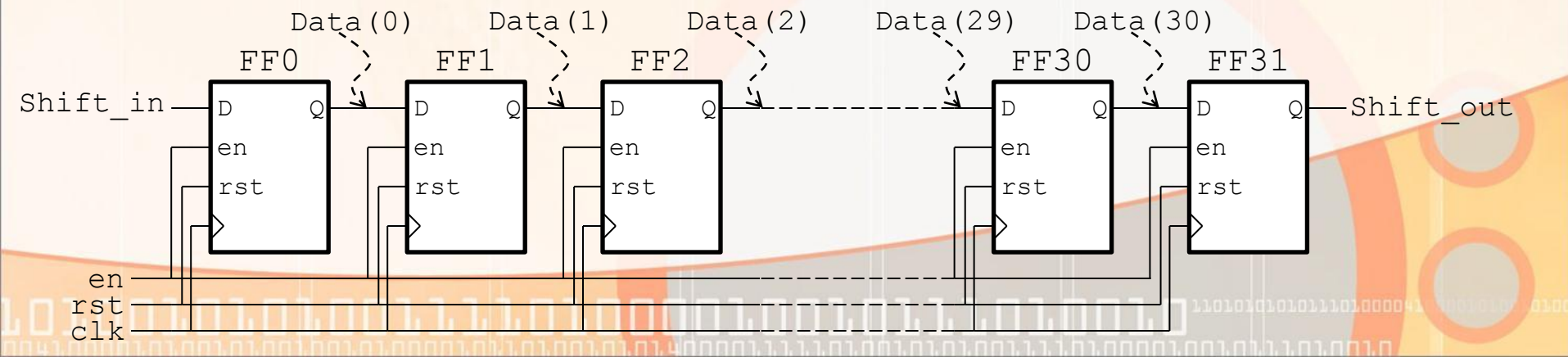
```
process (A)  
    variable I :  
        integer range (0 to 4) := 0;  
begin  
    Z <= "0000";  
    L1: loop  
        exit L1 when I = 4;  
        if (A = I) then Z(I) <= '1';  
        end if;  
        I := I + 1;  
    end loop;  
end process;
```

The For-Generate Loop

- The `for-generate` is a concurrent loop structure (i.e. it is defined outside processes) specifically designed for the synthesis of well-patterned structures – in other words, to create a fixed number of copies of the same logic block
 - Typical examples include creating a multi-bit adder from full adders, of parity trees from XOR gates
- Any concurrent statement may be included in a `for-generate` loop (including another `for-generate` loop) – no processes are allowed!
 - However, component instantiations (which can contain processes) may be made within `for-generate` loops
- Some restrictions apply:
 - All objects created must be identical
 - The loop parameter must be discrete

Using the For-Generate Loop

- Most commonly, `for-generate` loops are applied to *components*
- For example, assume that you want to generate a 32-bit shift-register out of flip-flops
- We want to place 32 of these flip-flops in a row and define wires to connect the i^{th} input to the $(i-1)^{\text{st}}$ output.
- The first input should be wired to `shift_in` and the last output should be wired to `shift_out`. Clock, reset and enable are shared by all FFs.



Using the For-Generate Loop

- *First, define the basic component for the iteration (in this case, a D-type flip-flop)*

```
entity D_FF is
  port (CLK, D, RST, EN : in STD_LOGIC;
        Q : out STD_LOGIC);
end D_FF;

architecture arch of D_FF is
begin
  DFF: process (CLK)
  begin
    if (CLK'event and CLK='1') then
      if (RST = '1') then
        Q <= '0';
      elsif (EN = '1') then
        Q <= D;
      end if;
    end if;
  end process DFF;
end arch;
```


Using the For-Generate Loop

- We could then use 32 component instantiation statements

```
entity shreg32 is
  port (clk, shift_in, rst, en : in STD_LOGIC;
        shift_out : out STD_LOGIC);
end shreg32;

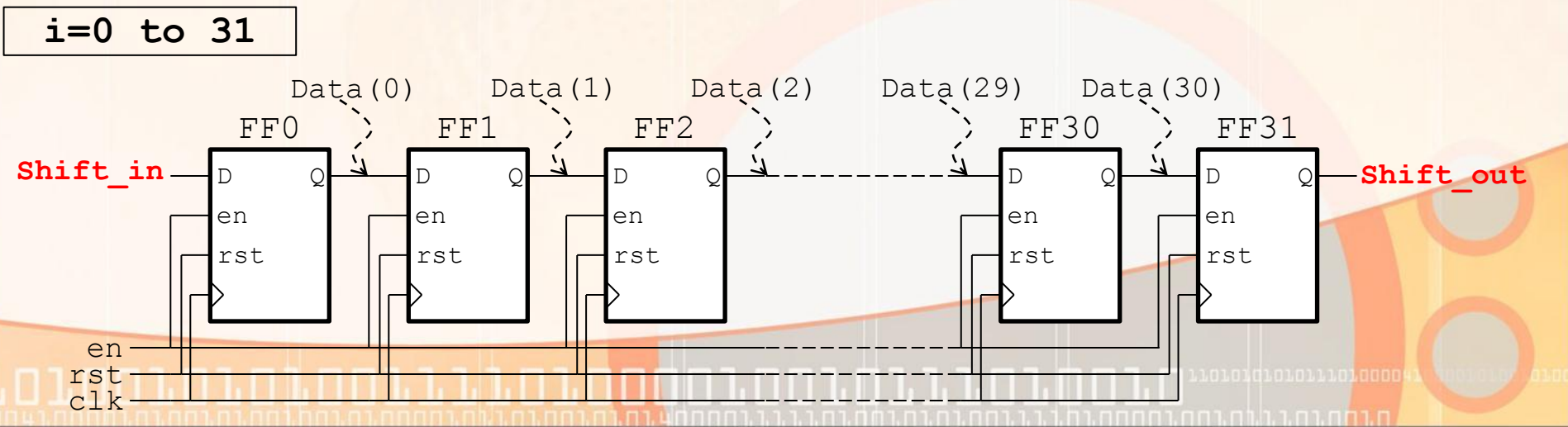
architecture arch of shreg32 is
  signal data : std_logic_vector(30 downto 0);
begin
  dff0: entity work.D_FF
    port map (CLK => CLK, D => shift_in, RST => RST,
              EN => EN, Q => data(0));
  dff1: entity work.D_FF
    port map (CLK => CLK, D => data(0), RST => RST,
              EN => EN, Q => data(1));
  [...] -- lots of D_FF component instantiations
  dff31: entity work.D_FF
    port map (CLK => CLK, D => data(30), RST => RST,
              EN => EN, Q => shift_out);
end arch;
```

Signals to carry data from one FF to the next (one less than the width of the register, since shift_in and shift_out are valid signals (ports))

Connect the output of one flip-flop to the input of the next

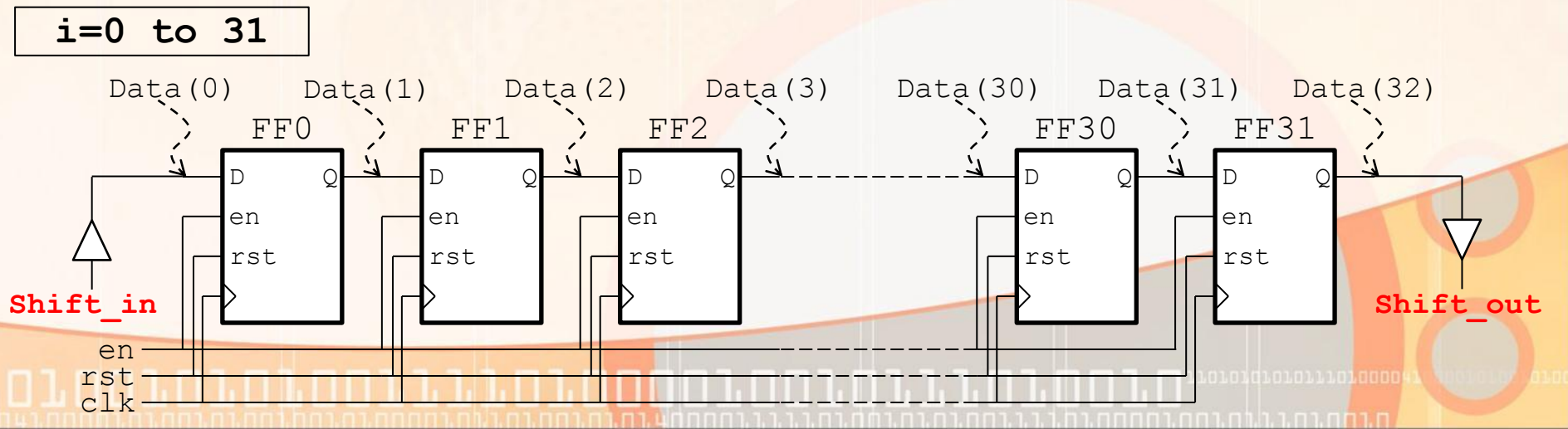
Using the For-Generate Loop

- And it would work. But of course it would be rather tedious. *The for-generate loop provides an alternative* by allowing the designer to create *i identical* components in a loop.
- To use a for-generate loop, however, *any differences between the components must be eliminated*.
- In this case, all FFs take their input from the *i-1*th and send their output to the *i*th line of the internal, (*i-1*)-line data bus, except the first and the last, which are connected to I/O ports



Using the For-Generate Loop

- Somehow, this difference must be avoided to preserve **consistency** across all iterations
- A very standard and common approach is to **extend the internal signals** to avoid the use of the I/O ports and preserve consistency
- In this case, the internal **data** bus can be extended (by two lines) so that all FFs take their input from the **i th** line and send their output to the **$i+1$ th** line



Using the For-Generate Loop

```
entity shreg32 is
  port (clk, shift_in, rst, en : in STD_LOGIC;
        shift_out : out STD_LOGIC);
end shreg32;

architecture arch of shreg32 is
  signal data : std_logic_vector (32 downto 0);

begin
  shift_reg: for i in 0 to 31 generate
    one_bit: entity work.D_FF
      port map (CLK => clk, D => data(i), RST => rst,
               EN => en, Q => data(i+1));
  end generate;

  data(0) <= shift_in;
  shift_out <= data(32);

end arch;
```

One bit more than the width of the register (to ensure consistency)

Unique label for each loop

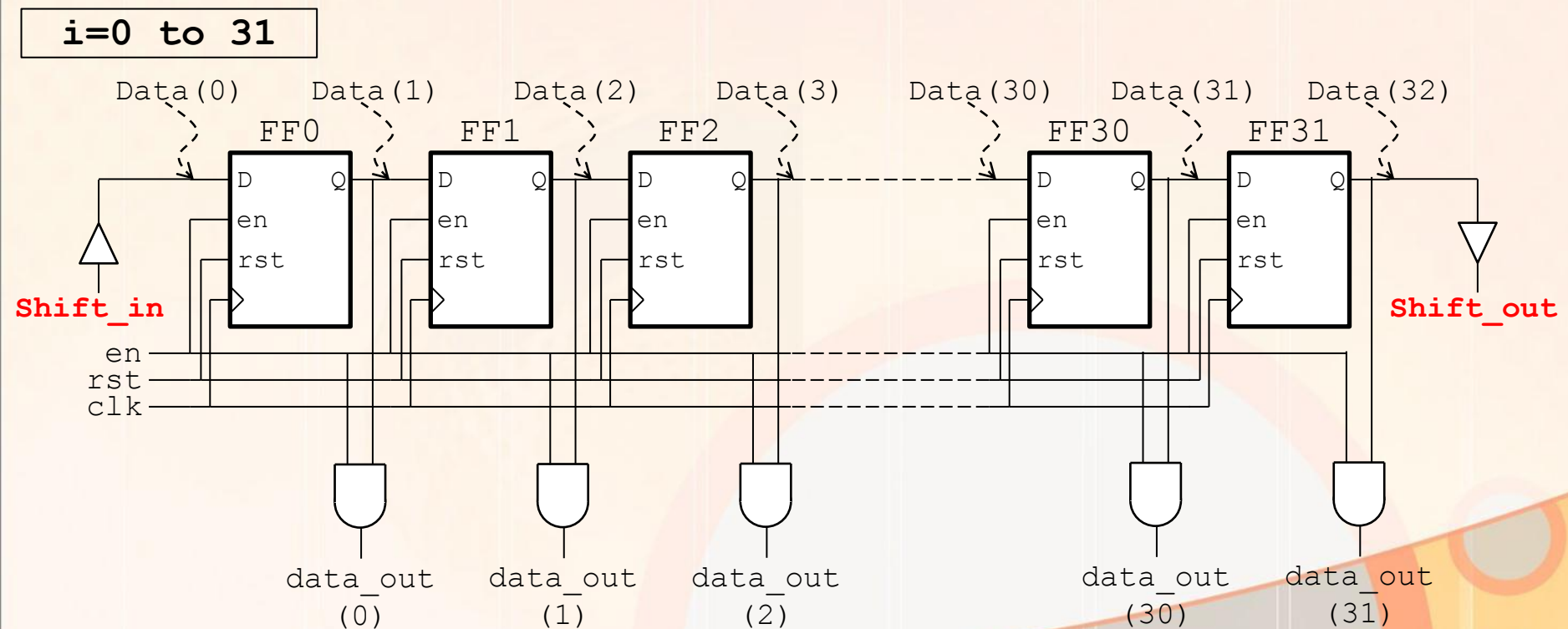
Generate 32 flip-flops

Connect the output of one flip-flop to the input of the next ... but look at the range of *i* !!!

Connect shift_in to first bit of data vector and last bit to shift_out (outside the loop)

Using the For-Generate Loop

- Note that the `for-generate` loop can also be used directly on combinational logic, or any *mix of combinational logic and components*, as long as all statements in the loop are concurrent (NO processes):

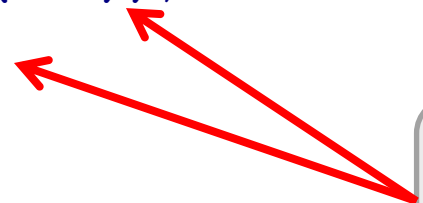


Using the For-Generate Loop

- Note that the `for-generate` loop can also be used directly on combinational logic, or any *mix of combinational logic and components*, as long as all statements in the loop are concurrent (NO processes):

```
entity shreg32 is
  port (clk, shift_in, rst, en : in STD_LOGIC;
        data_out : out STD_LOGIC_VECTOR(31 downto 0);
        shift_out : out STD_LOGIC);
end shreg32;

architecture arch of shreg32 is
  signal data : std_logic_vector (32 downto 0);
begin
  shift_reg: for i in 0 to 31 generate
    one_bit: entity work.D_FF
      port map (CLK => clk, D => data(i), RST => rst,
               EN => en, Q => data(i+1));
    data_out(i) <= data(i+1) AND en;
  end generate;
  data(0) <= shift_in;
  shift_out <= data(32);
end arch;
```



Any mix of
components
and logic

Parameterizable Iterative Structures

- Generics are very often used in conjunction with `for-generate` loops to generate “generic” components

```
entity shreg is
  generic (data_size : integer := 32);
  port (clk, shift_in, rst, en : in STD_LOGIC;
        shift_out : out STD_LOGIC);
end shreg;

architecture arch of shreg is
  signal data : std_logic_vector (data_size downto 0);

begin

  shift_reg: for i in 0 to data_size-1 generate
    one_bit: D_FF
      port map (CLK => clk, D => data(i), RST => rst,
               EN => en, Q => data(i+1));
  end generate;

  data(0) <= shift_in;
  shift_out <= data(data_size);
end arch;
```

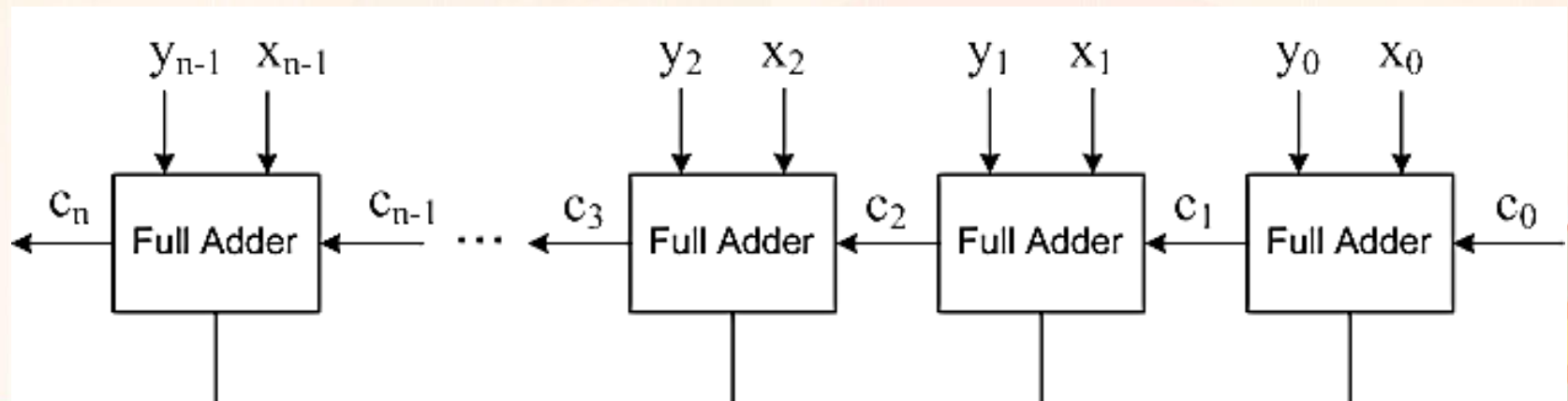
Default size of
shift register

Generates a shift
register of any size,
depending on the
value of `data_size`

Homework

- Write VHDL to generate an N bit adder from a full adder. N should be defined as *generic*.

```
entity full_adder is
    port (A,B,Cin: in std_logic;
          S, Cout: out std_logic);
end full_adder;
```



Thank you!

